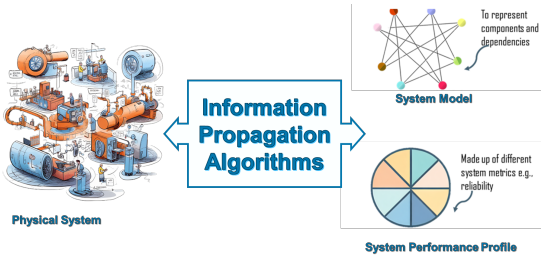
The Information Propagation Framework

A Julia toolkit for multi-metric analysis of DAG networks

What is the IPA Framework?



In one sentence

IPA is a **Julia-based** framework for analysing how information, failure, time, cost, and capacity propagate through networks represented as directed acyclic graphs.

- One graph representation reused across multiple analyses
- Exact DAG processing order from sources to sinks
- Same framework supports reliability, flow, and scheduling questions

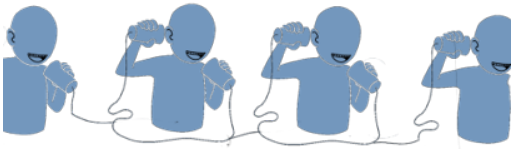
Core idea: process the DAG once, reuse everywhere

Two key ideas

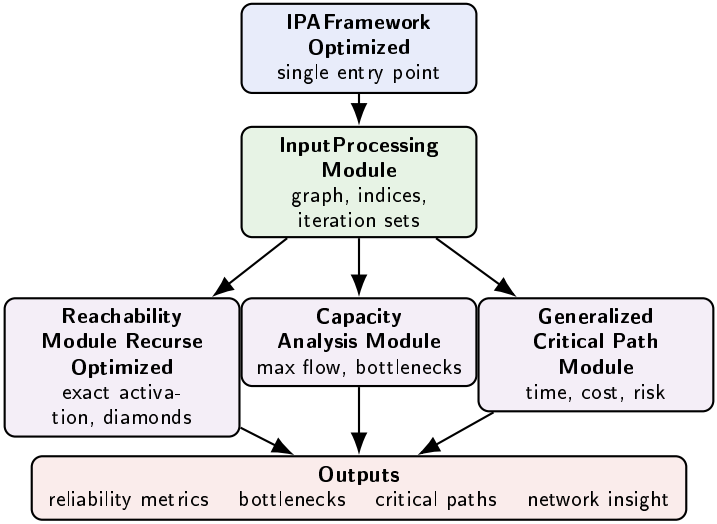
1. **Topological iteration sets** give a safe processing order
2. **Iterative message passing** propagates values layer by layer

Benefits

- No cyclic feedback issues
- Dependencies resolved before downstream nodes
- Natural parallelism within each layer
- For most core analyses: $O(V + E)$ time on a DAG

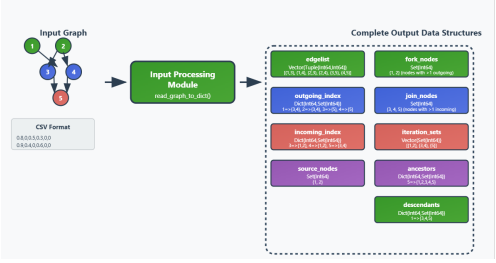


Architecture and dependency chain



The structural work is done once in the input layer, then reused by the analysis modules.

Input Processing layer: the shared graph object



Computed once:

- edge list
- incoming / outgoing indices
- source nodes
- iteration sets
- fork and join nodes
- ancestors / descendants

Why this matters

This turns later modules into **reusable analyses on one common DAG object**, instead of repeating graph preparation each time.

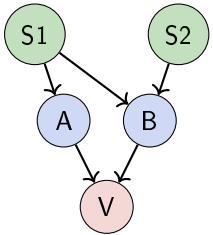
Reachability: what problem is it solving?

Goal

- Compute the exact probability that a node becomes active
- Activation means: **at least one causal path** from source nodes succeeds
- Avoid double-counting when several paths overlap upstream

Conceptually

$$P(v \text{ active}) = P(\text{at least one active source-to-}v \text{ path})$$



many paths can reach the target node

Main difficulty

The event is a **union of path events**, and those path events are often not independent.

Reachability math: regular multi-parent cases

Step 1: parent contributions

For parent i reaching child j through edge (i, j),

$$P_{i \to j} = P(i)\, P(i \to j)$$

Step 2: combine multiple parents

If parent-path events overlap, we need inclusion–exclusion:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$P\left(\bigcup_k A_k\right) \text{ extends similarly}$$

What the algorithm is really doing

- Process nodes in topological order
- Build activation from all upstream dependencies
- Use exact combination rules when paths overlap

Why not naive noisy-OR?

Noisy-OR is only safe when contributing events are independent. In DAGs with shared ancestry, that assumption can fail badly.

So the core task is exact probability of a union of dependent path events.

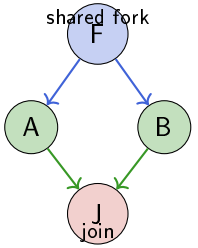
Reachability: the diamond problem

Diamond structure

- A fork sends influence down multiple branches
- Those branches reconverge later at a join
- The branches are not independent because they share the same fork ancestry

Why diamonds matter

- They break naive independence assumptions
- They are the main source of exact-conditioning work
- They explain why local graph structure matters mathematically



shared ancestry matters

$$P(\text{join}) = \sum_{s} P(\text{join} \mid s)\, P(s)$$

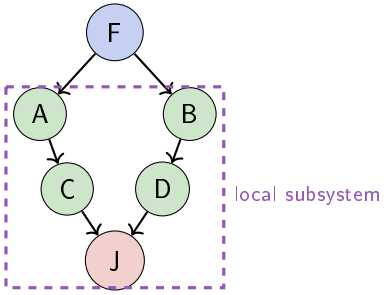
Diamond decomposition: simplify the exact computation

Decomposition idea

- Detect shared fork ancestry among parents of a join
- Group parents into diamond-related subsystems
- Extract only the local subgraph needed for exact conditioning

Why decompose?

- Simplifies reasoning inside complex DAGs
- Keeps exact maths local to the problematic region
- Makes nested diamond handling manageable
- Gives a natural subsystem for recursion and parallel work



Reachability representations: same maths, different uncertainty models

Three exact representations

- **Float:** one deterministic probability value
- **Interval:** lower and upper probability bounds
- **P-box:** richer uncertain probability description

Typical inputs

- node priors
- edge probabilities
- graph structure from input processing

Selection criteria

- **Need speed?** use Float
- **Need guaranteed bounds?** use Interval
- **Need richer uncertainty?** use P-box
- Trade-off is mainly between detail, memory, and runtime

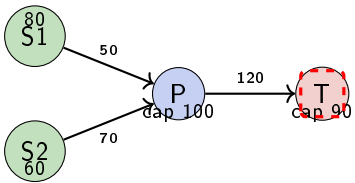
Key message

The reachability algorithmic logic is the same; what changes is the probability representation used by the arithmetic.

Capacity analysis: what is it solving?

Questions answered

- What throughput can the network sustain?
- Where is the bottleneck?
- How do node and edge limits interact?
- Which upgrades would most improve performance?



Core view

Capacity analysis is about **how much can propagate**, not just whether something can propagate.

Capacity analysis: maths, inputs, outputs, upgrades

Core equation

$$\mathrm{Flow}[v] = \min\left(\sum_{u \in \mathit{parents}(v)} \min(\mathrm{Flow}[u], c_{uv}),\ c_v\right)$$

Inputs

- node capacities
- edge capacities
- source input rates
- target nodes

Outputs

- total max flow
- bottleneck report
- path analysis / validation
- comparative or uncertain variants

Upgrade idea

Because the module identifies constrained edges and nodes, it can support upgrade recommendations such as **which component increase gives the biggest throughput gain**.

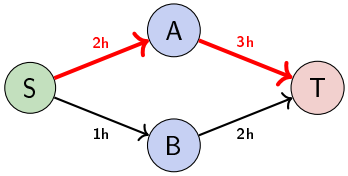
Critical path analysis: what is it solving?

Questions answered

- What is the longest or critical path?
- What is the earliest completion time?
- What is the maximum accumulated cost or risk?
- Which nodes or edges dominate the result?

Key idea

This is not restricted to time. The same pattern works for cost, risk, and other path-based metrics.



Critical path maths: a generalized three-stage engine

For each node in topological order:

Stage 1: $\mathrm{Propagate}$(parent value, edge value)

Stage 2: $\mathrm{Combine}$(all propagated parent values)

Stage 3: NodeFn(combined input, node value)

Analysis	Combine	Propagate	NodeFn
Time	max	$parent + edge$	$input + duration$
Cost	max	$parent + edge$	$input + cost$
Bottleneck	max	$\min(parent, edge)$	$\min(input, cap)$
Risk	max	$parent + edge$	$input + risk$

One generalized engine, many path-based analyses.

Critical path inputs, outputs, and extensibility

Typical inputs

- node durations / costs / risks
- edge delays / transition costs
- project start value
- chosen combination, propagation, and node functions

Outputs

- critical value at target nodes
- critical path nodes
- backward-pass or slack-style analysis

Why it is useful in the framework

It reuses the same DAG preprocessing and lets the user swap in different semantics without changing the graph logic.

Extension idea

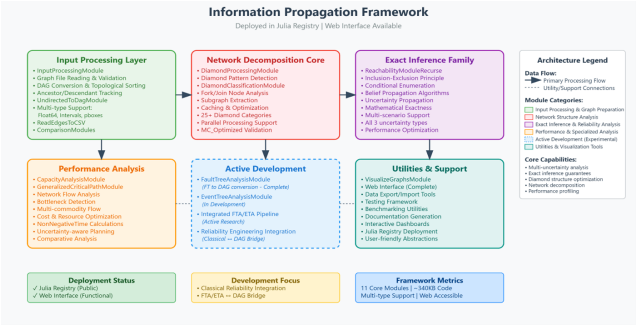
Any metric with a sensible path propagation rule can be inserted here: time, cost, risk, resilience score, or a custom domain-specific quantity.

Framework summary

Best fit

Projects where the network is fixed but the question changes: reliability, throughput, timing, cost, or resilience.

In short: preprocess once, analyse many times.



Thank you

Questions?